DOM

JS的三大组成部分：
ECMAScript.
DOM document object model=> 文档对象模型
BOM browser object model =>浏览器对象模型

nodeType常数值
1 元素节点
2 属性节点
3 文本节点
4 字符数据节点(文本不会被解析器解析)
5 实体引用节点
6 实体节点
7 处理指令节点
8 注释节点
9 文档节点(DOM树的根节点)
10 向为文档定义的实体提供接口
11 表示邻接节点和它们的子树。
12 代表一个符号在DTD中的声明

box.children

//返回第一级子节点【不包含文本或注释节点】box.children
// 返回一个类数组，类数组的取值操作：[]下标取值; children是一个属性，不是一个方法所以不用加（）；

box.children[1].nodeType //返回nodeType对应的常数值
box.children[0].nodeType

元素节点的节点类型nodeType;
obj.childNodes //获取到所有的子节点，包括文本节点
box.childNodes[0].nodeType //3

nodeName 返回节点名称

//children为类数组，类数组通常有length的方法；属于动态获取；只要里面孩子的个数变了，都能动态地反馈出来。

含文本或注释节点

box.childNodes[0].nodeName //#text
box.childNodes[1].nodeName //p 返回节点名称

tagName

box.childNodes[0].tagName
//如果为文本节点/注释节点是会返回undefined;

box.childNodes[1].tagName //p

firstChild

box.firstChild //返回第一个子节点【子节点包含文本/注释节点】 #text
box.firstElementChild //返回第一个子节点【子节点不包含文本/注释节点】 ...
box.lastChild //返回最后一个子节点，同上
box.lastElementChild //返回最后一个子节点，同上【子节点不包含文本/注释节点】

box.nextSibling //返回下一个兄弟节点;【子节点包含文本/注释节点】
box.nextElementSibling //返回下一个兄弟节点;【子节点不包含文本/注释节点】

parentNode offsetParent

box.parentNode //返回父节点;body
box.offsetParent //返回父节点;返回定位父级

box.childElementCount //返回第一级子元素的个数

```
<div id="box">
    <p summy='123'>
        <span>一顿操作猛如虎，一问工资两千五</span>
    </p>
    <span>123</span>
    <!-- 哈哈 -->
</div>




<script>

    /*
        JS的三大组成部分:
            ECMAScript.
            DOM  document object model  =>  文档对象模型
            BOM  browser object model => 浏览器对象模型
    */

    var box = document.getElementById('box');
    document.onclick = function(){}

    /*
        childNodes:
            text 文本节点
            标签节点
            comment  注释节点

                nodeType常数值
                    1   元素节点
                    2   属性节点
                    3   文本节点
                    4   字符数据节点(文本不会被解析器解析)
                    5   实体引用节点
                    6   实体节点
                    7   处理指令节点
                    8   注释节点
                    9   文档节点(DOM树的根节点)
                    10  向为文档定义的实体提供接口
                    11  表示邻接节点和它们的子树。
                    12  代表一个符号在DTD中的声明
```

```
        */

        console.log( box.childNodes )  //操作子节点
        console.log( box.children )   //p,span  返回第一级子节点【不包含文本或注释节点】
        console.log( box.children[1].nodeType )//1
        console.log( box.children[0].nodeType )//1
        console.log( box.childNodes[0].nodeType )//3
        console.log( box.childNodes[0].nodeName )//#text
        console.log( box.childNodes[1].nodeName ) //p  返回节点名称
        console.log( box.childNodes[0].tagName ) //如果为文本节点/注释节点是会返回undefined;
        console.log( box.childNodes[1].tagName ) //p

        console.log( box.getAttribute( 'id' ) )//box
        console.log( box.firstChild )  //返回第一个子节点【子节点包含文本/注释节点】 #text
        console.log( box.firstElementChild )  //返回第一个子节点【子节点不包含文本/注释节点】<p>...</p>
        // console.log( box.lastChild )    //返回最后一个子节点，同上
        // console.log( box.lastElementChild )   //返回最后一个子节点，同上

        console.log( box.nextSibling )  //返回下一个兄弟节点;【子节点包含文本/注释节点】
        console.log( box.nextElementSibling ) //返回下一个兄弟节点;【子节点不包含文本/注释节点】
        console.log( box.parentNode )   //返回父节点;body
        console.log( box.offsetParent )  //返回父节点;body

        console.log( box.childElementCount )  //返回第一级子元素的个数 2




    </script>
```

createElement/appendChild

创建节点

classList.add() 添加类名

classList.remove()删除类名

con.appendChild(p)//向节点中添加最后一个子节点(p要定义)

cloneNode

cloneNode() 方法可创建指定的节点的精确拷贝。

cloneNode() 方法 拷贝所有属性和值。

该方法将复制并返回调用它的节点的副本。如果传递给它的参数是 true，它还将递归复制当前节点的所有子孙节点。否则false，它只复制当前节点。

```
// cloneNode();
    //false 浅克隆、浅拷贝;
    //true  深克隆、浅拷贝;
var cloneCon = con.cloneNode( true );
console.log( cloneCon )
oDiv.appendChild( cloneCon )
// document.innerHTML = cloneCon
```

replaceChild

node.replaceChild(newnode,oldnode)

insertBefore

node.insertBefore('span,odiv') 把span元素添加到oDiv前面

BOM

browser object Mode 提供方法去操作浏览器的功能；

window对象

Screen 对象包含有关客户端显示屏幕的信息。

History 对象包含用户（在浏览器窗口中）访问过的 URL。

Location 对象包含有关当前 URL 的信息。(比如hash,host,hostname,href,origin;)

//警告框经常用于确保用户可以得到某些信息。
alert("文本")
//确认框用于使用户可以验证或者接受某些信息。
confirm("文本")
//提示框:经常用于提示用户在进入页面前输入某个值。
prompt("文本","默认值")

Navigator 对象包含有关浏览器的信息。

console.log(navigator.appName) //网景公司
console.log(navigator.cookieEnabled)//返回浏览器是否启用cookie的布尔值;

onblur

失去焦点

```
//页面切换，也可以获取光标或失去光标;
//当前页面不会被渲染;
//轮播会报错，因为切换到别的页面，页面不会渲染到，但是定时器还是会走。
var input = document.getElementsByTagName( 'input' )[0];
input.onfocus = function(){
console.log( '获得焦点' );
}
input.onblur = function(){
console.log( '失去焦点' )
}
//理论上是内容发生改变的时候才会触发;前提是得失去内容才能触发事件

//focus 聚焦于某个能获取焦点的对象;
//blur 让某个能获得焦点的对象失去焦点;
```

```
input.onblur = function(){
this.focus();
}
input.focus = function(){
this.blur();
}

input.onchange = function(){ console.log( this.value ); }
 //内容改变就会触发（能获取光标，），没有任何前提;
 //H5新事件，不兼容低版本IE浏览器.
```

键盘事件

```
//先后顺序:先down,再press,后：up;
 input.onkeydown = function(){
 console.log( '键盘按下' )
}
 input.onkeyup = function(){
console.log( '键盘抬起' )
 }
 input.onkeypress = function(){
console.log( '键盘按下事件' ) }

 /* onkeydown 键盘按下 (任何按键按下都会触发)
 onkeyup 键盘抬起（任何按键抬起都出触发）
 onkeypress 键盘按下事件 (只会相应字母键、数字键、符号键 / 不响应功能键F1等等 )
 onkeydown 与 onkeypress的区别： onkeypress 这个事件在用户按下并放开任何字母数字键时发生。系统按钮（例如，箭头键和功能键）无法得到识别。
onkeydown 这个事件在用户按下任何键盘键（包括系统按钮，如箭头键和功能键）时发生。 */
```

对应键的属性

keycod键盘码

```
input.onkeydown = function(e){
console.log( '键盘按下' )
console.dir( e )
}
```

```
ipt.onkeydown = function (e) {
    //同时按下Ctrl+alt+波浪符 ，弹出对应的内容
    if (e.ctrlKey && e.altKey) {
        this.value = ' hello wold! '
    }
}
```